

SUMMER TRAINING REPORT
ON
UDP-BASED SECURE P2P FILE SHARING AND CHAT APPLICATION
ON LOCAL AREA NETWORK

At



DEFENCE ELECTRONICS APPLICATIONS LABORATORY (DEAL)
DEFENCE RESEARCH & DEVELOPMENT ORGANISATION (DRDO)
MINISTRY OF DEFENCE, GOVERNMENT OF INDIA
DEHRADUN - 248008, INDIA

JULY, 2026

in partial fulfilment of the degree

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Graphic Era Hill University, Dehradun

Submitted By:
RISHAV SINGH NEGI

Under the Guidance of:
Mr. ANKUR AWASTI
Scientist, DEAL

DECLARATION

I hereby declare that the summer training report entitled "UDP-Based Secure P2P File Sharing and Chat Application on Local Area Network" submitted by me at Defence Electronics Applications Laboratory (DEAL), DRDO, Dehradun, is an authentic record of my training work carried out during the period of my internship under the supervision and guidance of Mr. Ankur Awasthi, Scientist, DEAL, DRDO.

Date: July 08, 2026

Place: Dehradun

SIGNATURE

**ANKUR AWASTHI
(SCIENTIST DEAL)**

ACKNOWLEDGEMENT

I express my deep gratitude and sincere thanks to my project guide, Mr. Ankur Awasthi, Scientist, Defence Electronics Applications Laboratory (DEAL), Dehradun, for his invaluable guidance, support, and mentorship throughout my training. His insights and feedback have been critical to the success of this project.

I would also like to express my sincere appreciation to the Director, DEAL, and the Head of Department/Division for providing me the opportunity to carry out my summer training at this esteemed laboratory and for allowing me to utilize the facilities.

Furthermore, I thank the faculty members of the Department of Computer Science and Engineering, Graphic Era Hill University, Dehradun, for their constant encouragement and academic support.

Finally, I extend my heartfelt thanks to all the scientists, staff members, and my peers at DEAL, DRDO, who helped me directly or indirectly during my internship.

RISHAV SINGH NEGI

ORGANIZATION PORTFOLIO

Defence Electronics Applications Laboratory (DEAL)

Defence Electronics Applications Laboratory (DEAL), Dehradun, is a premier Research and Development laboratory under the Defence Research and Development Organisation (DRDO), Ministry of Defence, Government of India.

Established with the vision of achieving self-reliance in electronic warfare and communication systems, DEAL is mandated to design and develop state-of-the-art technologies in the fields of:

- Satellite Communication Systems (SATCOM)
- Software Defined Radios (SDR) and Cognitive Radios
- Millimetre-Wave and Microwave Communication Links
- Secure Data Links for Unmanned Aerial Vehicles (UAVs)
- Radar Systems and advanced signal processing.

DEAL has made significant contributions to the Indian Armed Forces by delivering robust communication systems that operate under harsh electronic countermeasure environments. The laboratory is equipped with advanced testing facilities, simulation software, and specialized labs to foster cutting-edge research and product development.

TABLE OF CONTENTS

Declaration	ii
Acknowledgement	iii
Organization Portfolio	iv
List of Figures	vi
CHAPTER 1: INTRODUCTION	1
CHAPTER 2: LITERATURE SURVEY	8
CHAPTER 3: EXISTING SYSTEM	14
CHAPTER 4: PROPOSED SYSTEM	16
CHAPTER 5: SYSTEM ARCHITECTURE	19
CHAPTER 6: TECHNOLOGY STACK	21
CHAPTER 7: MODULE DESCRIPTION	23
CHAPTER 8: RELIABLE UDP PROTOCOL	27
CHAPTER 9: SLIDING WINDOW FLOW CONTROL	30
CHAPTER 10: TCP RENO INSPIRED CONGESTION CONTROL	31
CHAPTER 11: SECURITY IMPLEMENTATION	32
CHAPTER 12: COMPRESSION MECHANISM	33
CHAPTER 13: DATABASE DESIGN	35
CHAPTER 14: USER INTERFACE	35
CHAPTER 15: TESTING AND RESULTS	39
CHAPTER 16: FUTURE SCOPE	39
CHAPTER 17: CONCLUSION	40
REFERENCES	41

LIST OF FIGURES

Figure 7.1:	Peer Discovery Interface	22
Figure 7.2:	Secure Pairing Handshake Prompt	24
Figure 7.3:	Encrypted Peer-to-Peer Chat Room	25
Figure 7.4:	Live File Transfer & Telemetry Analytics	26
Figure 14.1:	Local File Transfer Logs & History	38

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid advancement of computer networks and communication technologies has significantly transformed the way information is exchanged between users and systems. Organizations, educational institutions, research laboratories, and enterprises continuously rely on efficient file-sharing mechanisms to collaborate and exchange digital information. Traditional methods of file transfer generally involve centralized servers, cloud-based storage services, or standard transport protocols such as TCP. Although these approaches provide reliable communication, they often introduce additional overhead, increased latency, dependency on internet connectivity, or require dedicated server infrastructure.

In many scenarios, especially within Local Area Networks (LANs), there exists a requirement for high-speed communication that minimizes overhead while maintaining reliability and security. Examples include research laboratories, defence establishments, universities, software development teams, and industrial environments where multiple users operate within the same network. In such environments, depending upon cloud infrastructure is unnecessary, while conventional file-sharing mechanisms may not provide the desired performance or flexibility.

User Datagram Protocol (UDP) provides a lightweight transport mechanism with minimal communication overhead. Unlike TCP, UDP does not establish a connection before transmitting data, making it faster and more suitable for latency-sensitive applications. However, UDP inherently lacks several essential features such as reliable delivery, packet ordering, retransmission of lost packets, congestion control, and flow control. Consequently, applications utilizing UDP must implement these mechanisms independently whenever reliable communication is required.

The primary objective of this project is to develop a secure peer-to-peer file-sharing application capable of utilizing UDP while implementing custom reliability mechanisms to achieve performance comparable to reliable transport protocols. The developed application, **SwiftShare LAN**, combines the efficiency of UDP with reliability features such as packet sequencing, acknowledgements, retransmissions, sliding window protocol, congestion control, encryption, compression, and automatic peer discovery.

SwiftShare LAN enables users connected to the same Local Area Network to discover each other automatically without requiring any centralized server. Users can communicate through encrypted messaging and transfer files securely using a custom Reliable UDP protocol. The application further integrates real-time monitoring capabilities that display various network statistics, including congestion window size, packet loss, transfer throughput, round-trip time (RTT), compression ratio, and transfer progress.

The project has been developed using modern software development technologies including Electron, React, Node.js, Express.js, Socket.IO, and native Node.js networking libraries. The graphical interface provides an intuitive desktop experience while the backend implements networking logic, transport-layer reliability, encryption, and data compression.

Unlike conventional file transfer applications, SwiftShare LAN provides a comprehensive networking platform that demonstrates practical implementation of several transport-layer concepts including congestion control algorithms inspired by TCP Reno, selective acknowledgements, retransmission mechanisms, packet integrity verification, secure communication using AES-256-GCM encryption, Brotli/Gzip compression, peer discovery using UDP Broadcast and Multicast, and visualization of network performance metrics.

The project therefore serves not only as a practical LAN communication tool but also as an educational demonstration of modern networking principles and transport protocol design.

1.2 Organization Profile

Defence Electronics Applications Laboratory (DEAL), DRDO

Defence Electronics Applications Laboratory (DEAL), Dehradun, is one of the premier laboratories functioning under the Defence Research and Development Organisation (DRDO), Government of India. The laboratory primarily focuses on the research, design, development, testing, and deployment of advanced communication systems for defence applications.

The laboratory contributes significantly towards strengthening India's defence communication infrastructure through indigenous development of secure communication technologies, satellite communication systems, tactical communication equipment, networking technologies, data communication protocols, electronic warfare support systems, and software-defined communication platforms.

DEAL actively works in various domains including:

- Tactical Communication Systems
- Satellite Communication
- Secure Data Communication
- Military Networking

- Embedded Communication Systems
- Wireless Communication
- Cryptographic Communication
- Network Management Systems
- Software Defined Radio (SDR)
- Defence Communication Protocols

The organization continuously develops indigenous technologies that enhance communication capabilities of the Indian Armed Forces while ensuring security, reliability, and interoperability among different defence communication platforms.

During the internship period, exposure was gained to several software engineering methodologies, networking concepts, transport protocols, communication architectures, and practical implementation techniques used in designing modern communication systems.

The internship provided valuable experience in applying theoretical networking concepts to solve real-world communication problems through software development and protocol implementation.

1.3 Problem Statement

Modern organizations generate and exchange enormous amounts of digital information every day. Existing file-sharing systems often depend on centralized servers, cloud infrastructure, or internet connectivity for transferring files between users. Although these approaches provide convenience, they introduce several limitations including dependency on external infrastructure, bandwidth bottlenecks, security concerns, and increased latency.

Within Local Area Networks, especially inside organizations, laboratories, educational institutions, and research facilities, users frequently require direct communication without relying on external servers. Conventional LAN file-sharing methods generally provide limited security, inefficient resource utilization, or lack intelligent congestion management mechanisms.

UDP provides an efficient transport protocol due to its connectionless nature and low communication overhead. However, it does not guarantee reliable delivery of packets. Packet loss, duplication, out-of-order arrival, and congestion remain major challenges while implementing file transfer using UDP.

Therefore, there exists a need to design and develop a secure peer-to-peer communication platform capable of:

- Discovering users automatically within a LAN.
- Providing encrypted communication.
- Ensuring reliable file transfer over UDP.
- Managing congestion dynamically.
- Minimizing bandwidth consumption.
- Providing real-time network performance visualization.

This project addresses these challenges through the implementation of a custom Reliable UDP protocol integrated with encryption, compression, congestion control, and peer discovery mechanisms.

1.4 Objectives

The primary objective of this project is to design and develop a secure, reliable, and efficient LAN-based peer-to-peer file-sharing application capable of providing high-speed communication using the User Datagram Protocol.

The major objectives include:

Primary Objectives

- Design a decentralized peer-to-peer communication platform.
- Implement automatic peer discovery using UDP Broadcast and Multicast.
- Develop a custom Reliable UDP protocol capable of reliable file transfer.
- Implement Sliding Window Flow Control.
- Implement TCP Reno-inspired congestion control.
- Develop packet acknowledgement and retransmission mechanisms.
- Provide packet sequencing and reassembly.
- Implement AES-256-GCM encryption for secure communication.
- Integrate Brotli and Gzip compression techniques.
- Design an intuitive desktop graphical interface.
- Provide secure messaging between users.

- Display live transfer analytics including throughput, RTT, congestion window, transfer speed, and packet loss.
 - Maintain transfer history and communication logs.
 - Ensure scalability for multiple users connected within the same Local Area Network.
-

Secondary Objectives

- Reduce communication overhead compared to TCP.
 - Improve bandwidth utilization.
 - Demonstrate implementation of transport-layer networking concepts.
 - Develop modular software architecture.
 - Enhance practical understanding of networking protocols.
 - Provide a platform suitable for future implementation of advanced networking features including NAT traversal, relay servers, resumable transfers, and adaptive congestion control.
-

The successful completion of these objectives demonstrates the practical implementation of modern networking concepts while providing an efficient, secure, and scalable file-sharing solution suitable for Local Area Network environments.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction

File sharing has become one of the fundamental services in modern computer networks. Whether in corporate organizations, educational institutions, defence establishments, or research laboratories, exchanging information quickly and securely is essential for effective collaboration. Numerous communication protocols and applications have been developed over the years to facilitate digital file transfer. However, each protocol has been designed with specific objectives, resulting in differences in terms of reliability, performance, scalability, security, and resource utilization.

Traditional protocols such as FTP, HTTP, and SMB primarily rely on the Transmission Control Protocol (TCP) to provide reliable communication. TCP guarantees packet delivery through acknowledgements, retransmissions, flow control, and congestion control mechanisms. Although TCP provides excellent reliability, these mechanisms introduce additional communication overhead, making it less suitable for scenarios requiring extremely low latency or high-speed communication over local networks.

On the other hand, the User Datagram Protocol (UDP) offers a lightweight communication model with significantly lower overhead. UDP does not establish a connection before data transmission, resulting in faster communication and reduced latency. However, UDP does not guarantee reliable packet delivery, ordering, or congestion management. Consequently, applications utilizing UDP must implement these functionalities independently whenever reliable communication is required.

Several modern applications such as online gaming, video conferencing, VoIP, live streaming, and multimedia broadcasting utilize UDP because of its low latency characteristics. These applications often implement custom reliability mechanisms tailored to their specific requirements.

The objective of this project is to combine the efficiency of UDP with reliability mechanisms inspired by TCP while maintaining higher flexibility and better performance within Local Area Networks.

2.2 Existing Technologies

Several communication protocols currently dominate computer networking applications.

2.2.1 File Transfer Protocol (FTP)

File Transfer Protocol is one of the oldest and most widely used protocols for transferring files between computers.

Advantages

- Reliable communication
- Supports authentication
- Standardized protocol
- Easy deployment
- Widely supported

Limitations

- Requires dedicated server
 - Plain-text communication in basic FTP
 - High protocol overhead
 - Not suitable for decentralized communication
 - Limited real-time monitoring
-

2.2.2 Hypertext Transfer Protocol (HTTP)

HTTP is primarily designed for transferring web resources but is frequently used for downloading files.

Advantages

- Universally supported
- Platform independent
- Easy integration

Limitations

- Client-server architecture
 - Internet dependency
 - Additional protocol overhead
 - Limited LAN optimization
-

2.2.3 Server Message Block (SMB)

SMB provides file and printer sharing services within Windows-based Local Area Networks.

Advantages

- Simple file sharing
- User authentication
- Access control

Limitations

- Windows dependent
 - Performance degradation on larger networks
 - Limited transport-layer customization
-

2.2.4 Cloud-Based File Sharing

Modern applications including Google Drive, OneDrive and Dropbox have revolutionized file sharing through cloud storage.

Advantages

- Global accessibility
- Automatic synchronization
- Backup capabilities

Limitations

- Internet dependency
 - Privacy concerns
 - Storage limitations
 - Subscription costs
 - Increased latency
-

2.3 TCP vs UDP

Both TCP and UDP operate at the Transport Layer of the TCP/IP protocol stack. Despite serving similar purposes, their characteristics differ significantly.

Feature	TCP	UDP
Connection	Connection-Oriented	Connectionless
Reliability	Guaranteed	Not Guaranteed
Packet Ordering	Yes	No
Retransmission	Automatic	Application Controlled
Congestion Control	Built-in	Not Available
Flow Control	Built-in	Not Available
Speed	Moderate	High
Latency	Higher	Lower
Overhead	High	Low
Suitable Applications	Web, FTP, Email	Gaming, Streaming, VoIP

TCP provides built-in mechanisms for reliability, whereas UDP prioritizes speed and simplicity. The goal of SwiftShare LAN is to bridge this gap by implementing TCP-like reliability while retaining the performance benefits of UDP.

2.4 Motivation Behind Reliable UDP

Although TCP performs exceptionally well over the Internet, it is often unnecessary to incur its full overhead in controlled Local Area Networks where packet loss is relatively low and communication latency is minimal.

A custom Reliable UDP protocol allows developers to selectively implement only those mechanisms required by the application.

Advantages include:

- Reduced communication overhead
- Better bandwidth utilization
- Lower latency
- Custom congestion control
- Application-specific optimization
- Fine-grained retransmission strategies

This flexibility motivated the development of SwiftShare LAN.

CHAPTER 3: EXISTING SYSTEM

3.1 Existing File Sharing Systems

Most existing file-sharing applications rely on centralized servers. Before communication begins, clients establish connections with remote servers responsible for authentication, storage, synchronization, and transfer management.

This architecture works efficiently over the Internet but becomes unnecessary inside Local Area Networks where users already share the same network infrastructure.

Several disadvantages of centralized systems include:

- Server dependency
 - Increased operational cost
 - Single point of failure
 - Additional latency
 - Network bottlenecks
 - Internet dependency
-

3.2 Limitations of Existing Systems

Current LAN communication systems suffer from multiple limitations.

Centralized Architecture

Most applications depend upon central servers.

High Protocol Overhead

TCP establishes a connection using a three-way handshake before data transmission.

Lack of Real-Time Analytics

Traditional applications rarely provide detailed transfer statistics such as:

- RTT
- Packet Loss
- Congestion Window
- Throughput
- Compression Ratio

Limited Security

Many LAN utilities rely only on operating system permissions without implementing end-to-end encryption.

Poor Scalability

Existing solutions often struggle when multiple simultaneous transfers occur.

CHAPTER 4: PROPOSED SYSTEM

4.1 Proposed Solution

SwiftShare LAN is designed as a decentralized peer-to-peer communication platform capable of discovering users automatically and establishing secure communication channels over Local Area Networks.

Instead of relying upon centralized servers, each node functions simultaneously as a client and a server.

The application architecture consists of four major components:

1. Desktop User Interface
2. Backend Communication Server
3. Reliable UDP Engine
4. Cryptographic & Compression Layer

These components work together to provide secure, reliable, and high-performance communication.

4.2 Key Features

The developed system incorporates several advanced networking features.

Automatic Peer Discovery

Users connected to the same LAN are discovered automatically using UDP Broadcast and Multicast packets.

Secure Pairing

Before communication begins, both users explicitly approve connection requests.

Reliable UDP

Unlike conventional UDP communication, the system implements:

- Packet numbering
- Acknowledgements
- Retransmission
- Packet ordering
- Flow control

Dynamic Sliding Window

The sender continuously adjusts its congestion window according to network conditions.

Benefits include:

- Improved throughput
- Better bandwidth utilization
- Reduced congestion
- Higher reliability

TCP Reno Inspired Congestion Control

The congestion control algorithm follows principles similar to TCP Reno.

Major phases include:

- Slow Start
- Congestion Avoidance
- Fast Recovery
- Timeout Recovery

This ensures efficient utilization of available bandwidth while preventing network congestion.

End-to-End Encryption

All communication is protected using AES-256-GCM.

Security features include:

- Confidentiality
 - Integrity
 - Authentication
 - Replay protection
-

Intelligent Compression

Before transmission, files are compressed using:

- Brotli
- Gzip

This reduces transmission time and bandwidth consumption.

Real-Time Dashboard

The application continuously visualizes network performance using interactive graphs displaying:

- Congestion Window
- RTT
- Throughput
- Packet Loss
- Compression Ratio
- Transfer Speed

These analytics help users understand network behaviour during file transfers.

CHAPTER 5: SYSTEM ARCHITECTURE

5.1 Overall System Architecture

SwiftShare LAN follows a modular layered architecture where each component performs a dedicated function while interacting with the remaining modules through clearly defined interfaces. The architecture has been designed to ensure modularity, scalability, maintainability, and high performance. Each layer is independent, allowing future enhancements without affecting the overall system.

The system consists of the following major layers:

1. Presentation Layer (Electron + React)
2. Communication Layer (Socket.IO)
3. Reliable UDP Engine
4. Compression Layer
5. Encryption Layer
6. Database Layer
7. Peer Discovery Service

The Presentation Layer provides a user-friendly graphical interface through which users can discover peers, send messages, transfer files, monitor transfer statistics, and configure application settings.

The Communication Layer handles all signaling-related operations such as connection establishment, transfer requests, metadata exchange, and peer synchronization using Socket.IO.

The Reliable UDP Engine is the core component of the application. It implements custom transport-layer reliability by introducing acknowledgements, retransmissions, sequence numbering, packet ordering, timeout handling, and congestion control over UDP.

The Compression Layer minimizes bandwidth utilization by compressing files before transmission. Depending upon the data characteristics, Brotli or Gzip compression is selected.

The Encryption Layer secures communication using AES-256-GCM encryption. Every communication session generates a unique symmetric encryption key through an ephemeral key exchange mechanism.

The Database Layer stores user preferences, communication history, transfer logs, and application configuration in lightweight JSON files.

The Peer Discovery Service continuously broadcasts and listens for discovery packets within the Local Area Network, allowing devices to detect each other automatically without manual configuration.

5.2 System Workflow

The overall workflow of SwiftShare LAN consists of the following sequence of operations:

Step 1: Application Startup

When the application starts, the Electron main process launches both the backend server and the React-based graphical user interface. The backend initializes the peer discovery service, UDP sockets, Socket.IO server, and local database.

Step 2: Peer Discovery

The discovery service broadcasts discovery packets over the LAN using UDP Broadcast and Multicast. Nearby instances of SwiftShare LAN receive these packets and update their list of available peers.

Step 3: Connection Request

A user selects an available peer and sends a connection request. The receiving user is notified through a graphical prompt and can either accept or reject the request.

Step 4: Secure Handshake

Once the request is accepted, both peers exchange public keys and derive a shared secret used for AES-256-GCM encryption.

Step 5: Metadata Exchange

Before transmitting the file, metadata such as filename, file size, checksum, compression method, and encryption parameters are exchanged through Socket.IO.

Step 6: Compression

The file is compressed using Brotli or Gzip depending on the selected compression strategy.

Step 7: Encryption

The compressed data is encrypted using AES-256-GCM to ensure confidentiality and integrity.

Step 8: Packetization

The encrypted file is divided into fixed-size packets. Each packet receives a unique sequence number, checksum, and header information.

Step 9: Reliable UDP Transfer

Packets are transmitted using UDP. The Reliable UDP Engine manages acknowledgements, retransmissions, congestion control, timeout detection, and packet ordering.

Step 10: File Reconstruction

The receiver verifies packet integrity, decrypts the received data, decompresses the content, reconstructs the original file, and stores it in the selected destination.

CHAPTER 6: TECHNOLOGY STACK

SwiftShare LAN has been developed using modern software technologies that provide high performance, scalability, and cross-platform compatibility.

Electron

Electron provides the desktop environment for the application by combining Chromium with Node.js. It enables development of native desktop applications using web technologies.

Advantages:

- Cross-platform support
 - Native desktop interface
 - Access to operating system APIs
 - Integration with Node.js
-

React

React is used for developing the graphical user interface.

Features:

- Component-based architecture
 - Virtual DOM
 - Efficient rendering
 - Responsive user interface
-

Node.js

Node.js executes the backend networking logic.

Responsibilities:

- UDP communication
 - Socket.IO server
 - File management
 - Encryption
 - Compression
-

- Peer discovery
-

Express.js

Express provides REST APIs and backend service management.

Socket.IO

Socket.IO acts as the control channel between peers.

It is responsible for:

- Connection requests
 - User authentication
 - Metadata exchange
 - Chat messages
 - Transfer control
 - Status synchronization
-

UDP

UDP serves as the transport protocol for actual file transmission.

Reasons for choosing UDP:

- Low latency
 - Minimal protocol overhead
 - Better LAN performance
 - Custom reliability implementation
-

AES-256-GCM

AES-256-GCM provides authenticated encryption.

Benefits include:

- High security
 - Fast encryption
-

- Authentication tag
- Integrity verification

Brotli & Gzip

Compression minimizes bandwidth utilization and improves transfer speed.

CHAPTER 7: MODULE DESCRIPTION

SwiftShare LAN consists of several independent modules working together.

7.1 Peer Discovery Module

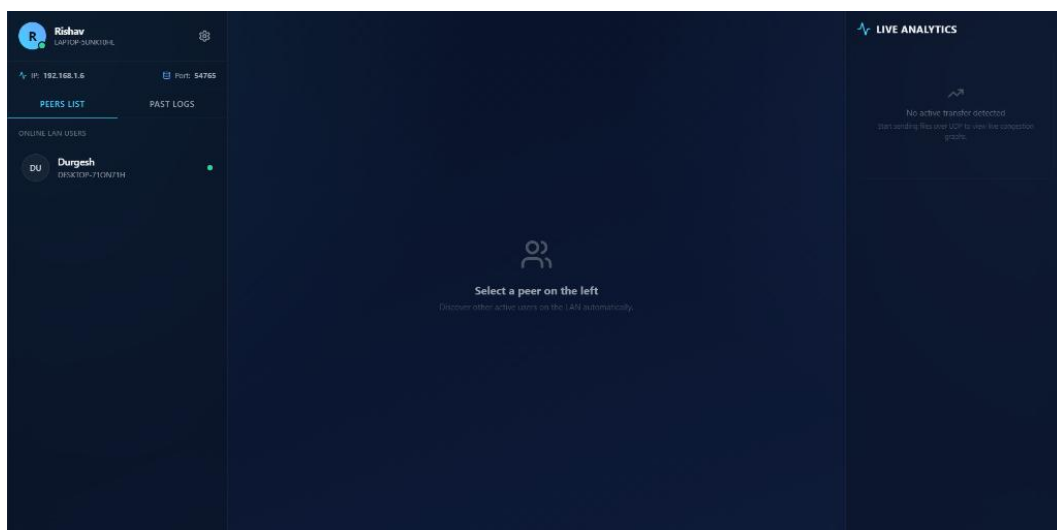
The Peer Discovery Module continuously scans the Local Area Network.

Responsibilities include:

- Broadcasting discovery packets
- Receiving discovery packets
- Maintaining active peer list
- Removing inactive users
- Updating online status

Discovery occurs automatically every few seconds, ensuring that newly connected users become visible without manual refresh.

Figure 7.1: Peer Discovery Interface



This interface displays all active devices connected to the same Local Area Network that are running SwiftShare LAN. The application automatically discovers nearby peers using UDP broadcast and multicast without requiring manual IP configuration. Users can select any available peer to initiate secure communication or file transfer.

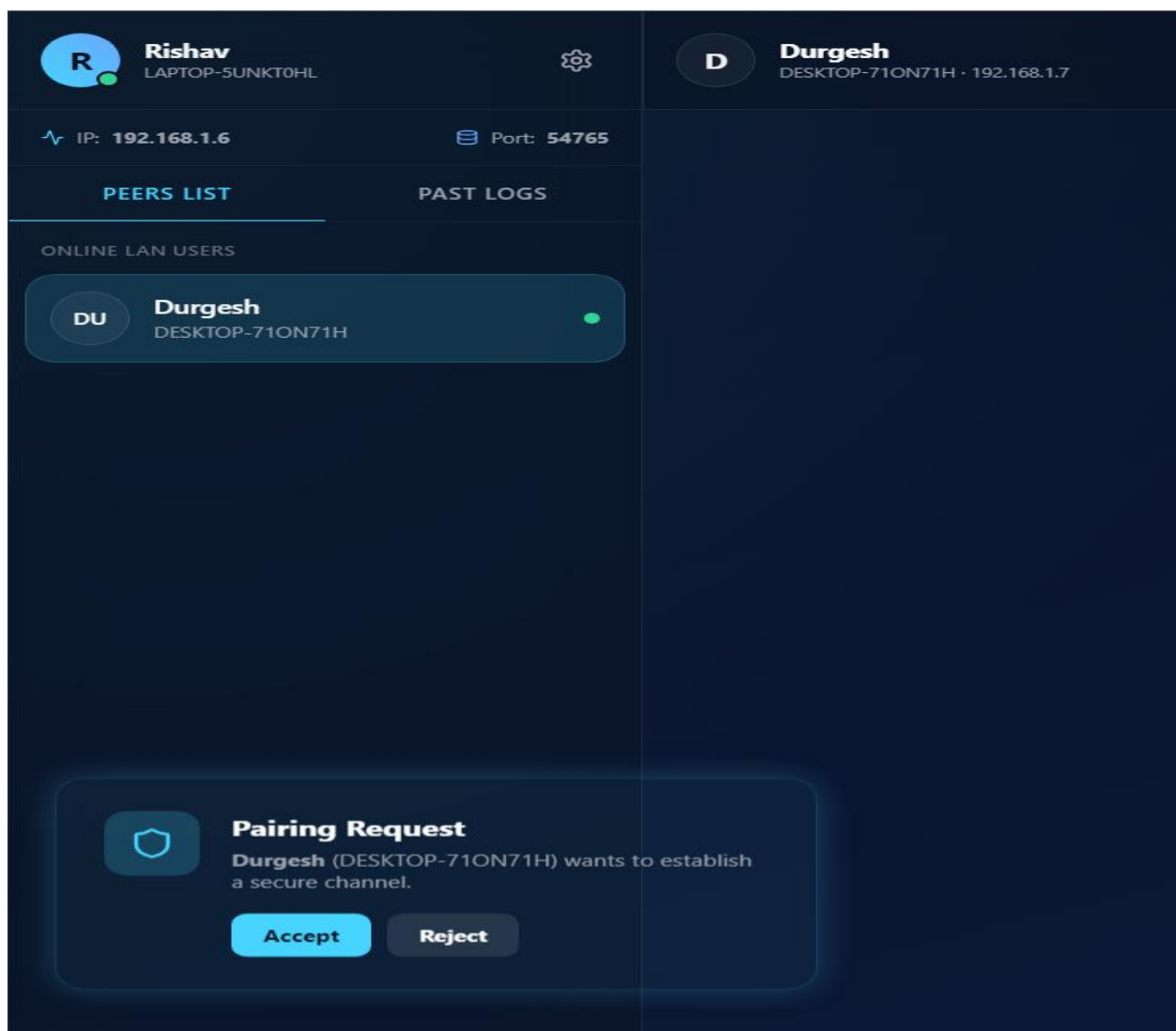
7.2 Authentication Module

Although SwiftShare LAN operates inside a Local Area Network, unauthorized communication is prevented through explicit connection approval.

The receiver must manually approve every communication request before data exchange begins.

This mechanism prevents unsolicited file transfers.

Figure 7.2: Secure Pairing Handshake Prompt



This interface displays the secure pairing handshake alert that appears on the receiver system when a peer tries to connect. To protect privacy and prevent spam, the user must explicitly accept or reject the connection request. Accepting the request triggers the Diffie-Hellman key exchange to secure future interactions.

7.3 Chat Module

The chat module enables encrypted text communication between connected users.

Features include:

- Real-time messaging
 - AES encryption
 - Timestamped messages
 - Session management
-

Figure 7.3: Encrypted Peer-to-Peer Chat Room



The chat window enables secure text communication between connected peers. Every text message sent is encrypted locally using the Derived AES-256-GCM symmetric session key before being transmitted over the LAN. Real-time message status indicators (such as seen double checkmarks) are updated automatically.

7.4 File Transfer Module

This module performs the actual file transmission.

Major responsibilities include:

- File selection
- Metadata generation
- Compression
- Encryption
- Packetization
- Reliable transmission
- Progress monitoring

- File reconstruction

7.5 Analytics Dashboard

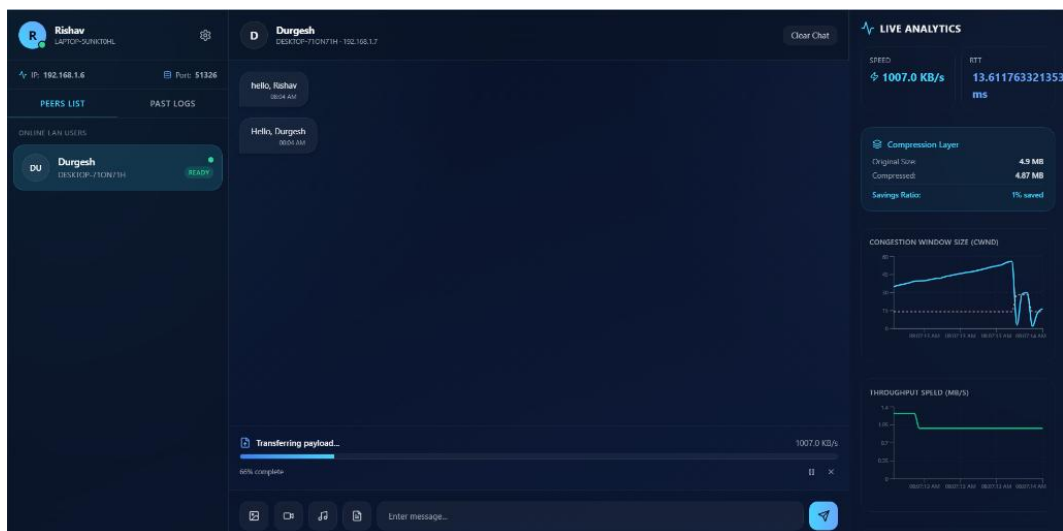
One of the unique features of SwiftShare LAN is its live monitoring dashboard.

The dashboard continuously displays:

- Congestion Window
- Round Trip Time
- Packet Loss
- Throughput
- Compression Savings
- Transfer Progress
- Transfer Speed

These statistics help visualize network behaviour during communication.

Figure 7.4: Live File Transfer & Telemetry Analytics



This module visualizes live transfer stats dynamically. Real-time telemetry graphs plot the Congestion Window (cwnd) sizes matching TCP Reno behaviour (e.g. multiplicative decrease during packet drops), smoothed round-trip time (RTT), throughput speeds in MB/s, and compression savings ratio.

CHAPTER 8: RELIABLE UDP PROTOCOL

The Reliable UDP Engine forms the backbone of SwiftShare LAN.

Unlike conventional UDP communication, which provides only best-effort delivery, this engine introduces several reliability mechanisms.

The protocol includes:

- Packet numbering
- Packet acknowledgement
- Timeout detection
- Retransmission
- Packet ordering
- Flow control
- Congestion control
- Packet integrity verification

Each outgoing packet contains:

- Packet Type
- Transfer ID
- Sequence Number
- Payload Length
- SHA-256 Checksum
- Encrypted Payload

After transmitting a packet, the sender waits for an acknowledgement.

If an acknowledgement is not received before the timeout expires, the packet is retransmitted.

This mechanism guarantees reliable delivery without relying on TCP.

8.1 Packet Ordering

UDP packets may arrive out of sequence.

The receiver temporarily stores out-of-order packets inside a buffer until all missing packets arrive.

Only after complete reconstruction is the file delivered to the user.

This prevents file corruption.

8.2 Packet Integrity

Every transmitted packet contains a SHA-256 checksum.

The receiver recalculates the checksum and compares it with the transmitted value.

If the values differ, the packet is considered corrupted and retransmission is requested.

8.3 Timeout Handling

Each transmitted packet starts an internal timer.

If acknowledgement is not received within the calculated Retransmission Timeout (RTO), the sender retransmits the packet.

Adaptive timeout calculation improves overall network performance.

CHAPTER 9: SLIDING WINDOW FLOW CONTROL

9.1 Introduction

Flow control is an essential mechanism in any reliable communication protocol. If packets are transmitted continuously without considering the receiver's processing capability or current network conditions, packet loss, congestion, and retransmissions increase significantly. To address this challenge, SwiftShare LAN implements a **Sliding Window Flow Control** mechanism inspired by the Transmission Control Protocol (TCP).

The sliding window controls the number of outstanding packets that can remain unacknowledged at any given instant. Instead of waiting for an acknowledgement after transmitting every individual packet, multiple packets are transmitted together within the current congestion window. This significantly improves throughput while maintaining reliability.

Initially, the congestion window (cwnd) is set to one packet. As acknowledgements are received successfully, the window expands, allowing more packets to be transmitted simultaneously. Whenever packet loss or timeout occurs, the congestion window is reduced to prevent network congestion. This adaptive behavior enables efficient utilization of available bandwidth while avoiding excessive retransmissions.

Advantages of Sliding Window

- Improves network throughput.
 - Minimizes idle transmission time.
 - Utilizes available bandwidth efficiently.
 - Supports multiple outstanding packets.
 - Reduces overall transfer duration.
 - Enables adaptive congestion management.
-

CHAPTER 10: TCP RENO INSPIRED CONGESTION CONTROL

10.1 Introduction

Although UDP does not provide congestion control, transmitting packets without considering network conditions may lead to severe congestion, increased packet loss, and degraded performance. Therefore, SwiftShare LAN incorporates a congestion control algorithm inspired by **TCP Reno** to dynamically regulate packet transmission.

The congestion control mechanism continuously monitors acknowledgements, retransmissions, timeout events, and round-trip time to determine the optimal transmission rate.

The implementation consists of four major phases.

10.2 Slow Start

Initially, the congestion window is maintained at a small value. For every successful acknowledgement received, the congestion window increases rapidly. This exponential growth continues until the predefined threshold is reached.

The objective of this phase is to quickly determine the available network capacity while avoiding unnecessary delays.

10.3 Congestion Avoidance

Once the slow start threshold is crossed, the congestion window increases gradually rather than exponentially. Linear growth prevents sudden congestion and allows efficient utilization of available bandwidth.

10.4 Fast Retransmission

Whenever duplicate acknowledgements indicate packet loss, the sender retransmits only the missing packet without waiting for the retransmission timeout. This considerably improves transfer efficiency by avoiding unnecessary delays.

10.5 Fast Recovery

Instead of restarting communication from the beginning after every packet loss, Fast Recovery reduces the congestion window moderately and resumes communication immediately. This prevents drastic reductions in throughput while maintaining network stability.

Benefits

- Higher throughput
- Reduced packet loss
- Faster recovery

- Efficient bandwidth utilization
 - Improved network stability
-

CHAPTER 11: SECURITY IMPLEMENTATION

11.1 AES-256-GCM Encryption

Security is one of the primary objectives of SwiftShare LAN. Since files may contain confidential information, all communication is encrypted before transmission.

SwiftShare LAN employs **AES-256-GCM (Advanced Encryption Standard with Galois/Counter Mode)**.

AES-256-GCM provides:

- Confidentiality
- Authentication
- Integrity Verification
- Protection against modification attacks

For every communication session, a unique symmetric key is generated through secure key exchange. Every transmitted packet is encrypted before packetization.

Even if an attacker intercepts the UDP packets, reconstructing the original data without the session key is computationally infeasible.

Advantages

- High security
 - Fast encryption speed
 - Low computational overhead
 - Authentication tag verification
 - Industry-standard encryption
-

11.2 SHA-256 Integrity Verification

Every packet generated by the Reliable UDP Engine contains a SHA-256 checksum.

Upon reception, the checksum is recalculated.

If both values match, the packet is accepted.

Otherwise, retransmission is requested.

This guarantees that corrupted packets never become part of the reconstructed file.

CHAPTER 12: COMPRESSION MECHANISM

Large files require considerable bandwidth and increase transmission time. To improve efficiency, SwiftShare LAN compresses files before encryption and transmission.

The application supports two compression algorithms.

Brotli

Brotli generally achieves higher compression ratios for text-based files.

Advantages include:

- Better compression
- Reduced bandwidth consumption
- Smaller transmission size

Gzip

Gzip provides faster compression while maintaining reasonable compression efficiency.

Suitable for:

- Binary files
- Multimedia files
- Large datasets

The sender automatically compresses the selected file before encryption. After successful reception, the receiver decompresses the file to restore the original content.

Benefits

- Reduced transfer time
 - Lower network utilization
 - Improved throughput
 - Faster communication
-

CHAPTER 13: DATABASE DESIGN

SwiftShare LAN maintains lightweight application data using JSON files.

The database stores:

- User Profile
- Device Information
- Chat History
- Transfer Logs
- User Preferences
- Network Configuration

The database uses atomic write operations to prevent corruption during unexpected application termination.

Advantages include:

- Simple implementation
 - Easy maintenance
 - Lightweight storage
 - Fast access
 - Platform independence
-

CHAPTER 14: USER INTERFACE

A modern graphical user interface has been developed using React and Electron.

The interface consists of the following modules:

Dashboard

Displays:

- Connected users
- Current transfer
- Network information

Chat Window

Provides secure communication between connected peers.

Features:

- Real-time messaging
- Encryption
- Message history

File Transfer Window

Displays:

- Selected file
- Progress bar
- Transfer speed
- Estimated completion time

Analytics Dashboard

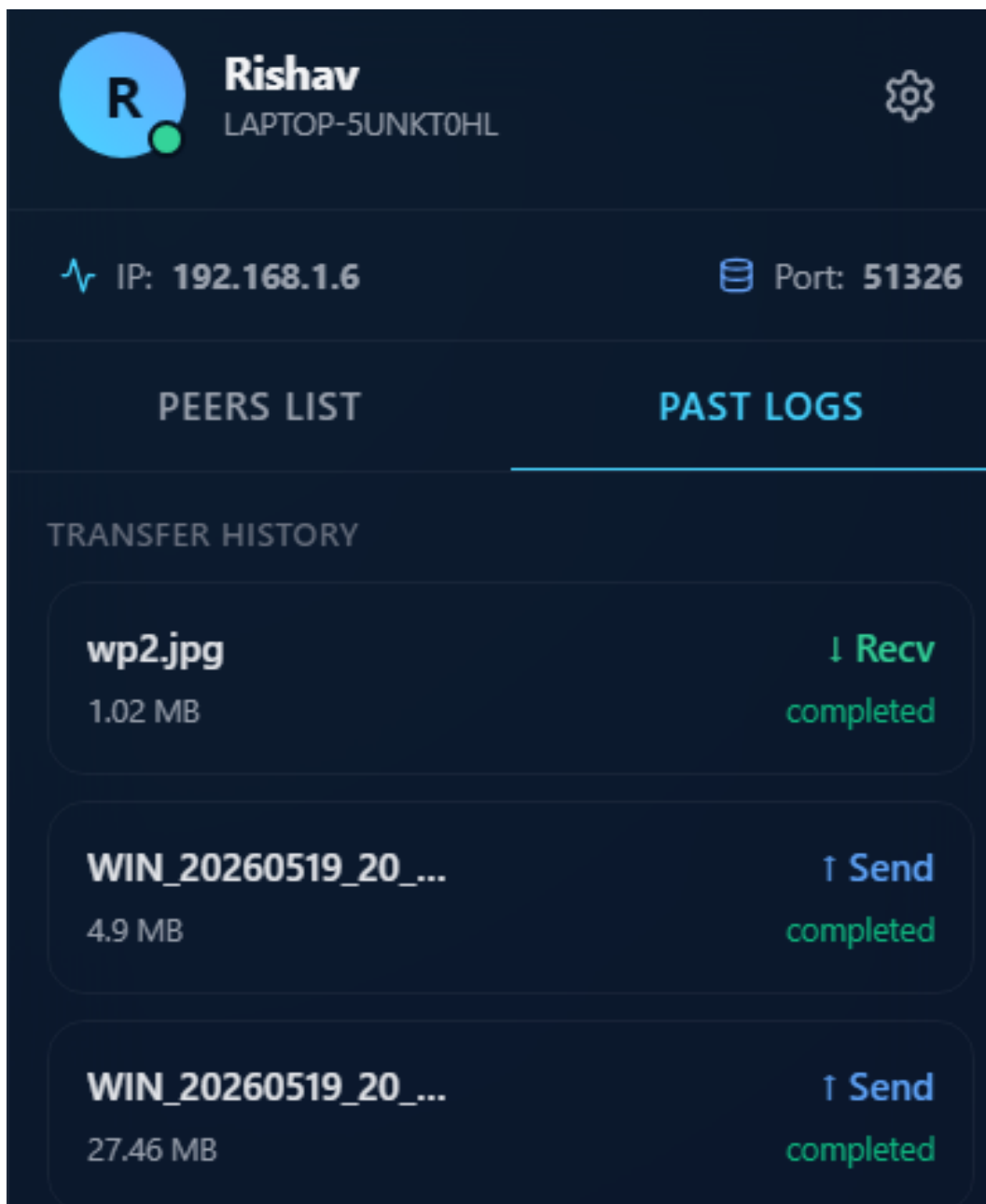
Visualizes:

- Congestion Window
- RTT
- Throughput
- Compression Ratio
- Packet Loss
- Transfer Speed

Transfer History

Maintains previous communication records.

Figure 14.1: Local File Transfer Logs & History



The Past Logs section lists all historical transfers conducted during the application's uptime. Users can view transfer directions (uploads/downloads), exact file sizes, and final execution states (completed, cancelled, or failed) recorded atomically in the local JSON database.

CHAPTER 15: TESTING AND RESULTS

The developed application was tested extensively over a Local Area Network using multiple systems.

The following functionalities were verified successfully:

- Automatic Peer Discovery
- Connection Establishment
- Secure Pairing
- Chat Communication
- File Transfer
- Compression
- Encryption
- Packet Retransmission
- Congestion Window Adjustment
- Throughput Visualization
- Transfer Logging

The testing demonstrated that the Reliable UDP protocol successfully transmitted files despite simulated packet loss. The congestion control mechanism dynamically adjusted the transmission rate according to network conditions.

Compression reduced transmission size for text-based files, while AES encryption ensured secure communication without noticeable performance degradation.

CHAPTER 16: FUTURE SCOPE

Although SwiftShare LAN provides a complete LAN communication platform, several improvements may be incorporated in future versions.

Future enhancements include:

- Cross-platform mobile application
- Internet-based communication
- NAT traversal
- Relay server support

- Resumable file transfers
- Differential synchronization
- Voice communication
- Video conferencing
- End-to-end authentication certificates
- AI-based congestion prediction
- Machine learning assisted bandwidth estimation
- Multi-threaded packet scheduling
- Distributed file sharing
- Cloud synchronization
- Automatic update mechanism

These enhancements would significantly increase the scalability and usability of the application.

CHAPTER 17: CONCLUSION

SwiftShare LAN successfully demonstrates the implementation of a secure and reliable peer-to-peer file-sharing platform using the User Datagram Protocol. The project combines the speed advantages of UDP with custom reliability mechanisms such as acknowledgements, retransmissions, packet sequencing, sliding window flow control, and TCP Reno-inspired congestion control.

The application further enhances communication security using AES-256-GCM encryption and SHA-256 integrity verification while improving efficiency through Brotli and Gzip compression. Automatic peer discovery, secure messaging, live analytics, and transfer monitoring provide a user-friendly experience while exposing important networking concepts through practical implementation.

The project has significantly strengthened understanding of computer networking, transport layer protocols, congestion control algorithms, software engineering principles, and secure communication systems. It also demonstrates how theoretical concepts learned during academic studies can be translated into a real-world software application.

Overall, SwiftShare LAN achieves its primary objective of providing secure, reliable, and efficient LAN-based file sharing while serving as a practical implementation of advanced networking techniques.

REFERENCES

1. W. Richard Stevens, *TCP/IP Illustrated, Volume 1*, Addison-Wesley.
2. Andrew S. Tanenbaum and David J. Wetherall, *Computer Networks*, Pearson Education.
3. Behrouz A. Forouzan, *Data Communications and Networking*, McGraw-Hill.
4. RFC 768 – User Datagram Protocol (UDP).
5. RFC 793 – Transmission Control Protocol (TCP).
6. RFC 5681 – TCP Congestion Control.
7. RFC 6298 – Computing TCP Retransmission Timer.
8. Node.js Documentation.
9. Electron Documentation.
10. React Documentation.
11. Socket.IO Documentation.
12. AES (FIPS-197) Standard Documentation.
13. Brotli Compression Specification.
14. Gzip Compression Specification.
15. DRDO and DEAL technical resources relevant to secure communication systems.